

GUÍA RESUMEN — Diseño de Features

Módulo 02 · Clean Architecture + Flutter + Supabase Resumen integral para diseñar **cualquier feature** en ~20 minutos, sin volver a leer el módulo completo. Hilo conductor: **Feature “Buyers”** (organizador gestiona compradores que reservaron tickets en sus rifas).

0 · Flujo de trabajo general

FADER —> MApeo —> Contratos —> Flujo —> Estimación
(10 min) (3 min) (3 min) (3 min) (2 min)

Regla de oro: “1 hora de papel, 30 min de código.” Diseña completo antes de abrir el editor.

Paso	Qué produces	Dónde aprender más
1. FADER	Feature descompuesta en piezas atómicas	01-descomposicion-feature.md
2. Mapeo	Cada pieza → una capa de Clean Arch	02-mapeo-capas.md
3. Contratos	Interfaces de cada capa (qué, no cómo)	03-contratos-primerio.md
4. Flujo	Diagrama de datos en forma de U	04-flujo-datos.md
5. Estimación	Tiempo con framework FADE + buffer	15-estimacion-complejidad.md

1 · FADER — Descomponer el problema (10 min)

Formular → **Actorizar** → **Descomponer** → **Entidades** → **Reglas**. De lo general a lo específico · De lo externo a lo interno. **Iterativo:** 1er pase rápido (15 min) → revisión cruzada → 2º pase (10 min).

F — Formular

Formato: “Como [actor], quiero [acción] para [valor].”
× “Hay que hacer un carrito.” · “Como cliente, quiero ver el total actualizado del carrito para saber cuánto gastaré.”

Preguntas guía: ¿Qué necesidad real resuelve? ¿Qué pasa si NO existe? ¿Quién se beneficia? ¿Cómo se usa hoy sin esta feature? ¿Qué expectativas tiene el usuario?

A — Actorizar

Tipo	Definición	Ejemplo
Usuario primario	Inicia la acción	Cliente que compra
Usuario secundario	Provee soporte	Admin que gestiona inventario
Sistema externo	API, servicio, BD	Supabase, pasarela de pagos
Sistema interno	Otra feature de la app	Módulo de tickets, notificaciones

Reglas: cada actor tiene razón de ser (no inventes) · un actor puede ser persona o sistema · un usuario puede ser varios actores.

D — Descomponer

Test de atomicidad (las 3 deben pasar):

#	Prueba	Pregunta	Si falla...
1	Un solo verbo	¿Hace una sola cosa?	Dívide
2	Un solo resultado	¿Un único cambio en el sistema?	Dívide
3	Valor independiente	¿Tiene sentido sola para el usuario?	Combínala o elimínala

Clasificación: [C] reate · [R] ead · [U] pdate · [D] elete · Validación · Cálculo · Transición.

Granularidad: pequeña 3-6 ops · mediana 7-15 ops · grande 16-30 ops · más de 20 ops → dividir en features. Si el nombre tiene “y”, divídelo.

E — Entidades

- Conceptos del **mundo real** (no tablas ni clases).
- Solo atributos **esenciales**.

- Pregunta clave: ¿existe en el negocio o solo en tu BD? `Producto` · × `ProductoTable` (eso vive en DATA).
- Define relaciones y qué NO es atributo de cada entidad.

R — Reglas

Categoría	Ejemplo
Restricción	Máx. 50 items por carrito
Cálculo	Impuesto 16% del subtotal
Validación	Cupón no expirado
Flujo	Si stock < cantidad → error
Consistencia	Descuento máx. 50% del total

Formato: `R001` + descripción + actor + severidad + mensaje de error. Si crees que una regla es “obvia”, escríbela igual.

Ejemplo real — Feature Buyers

```
[F] F1: Como organizador de rifas, quiero ver los últimos 30 compradores para saber quién me reservó.
F2: Como organizador, quiero buscar por nombre o teléfono (coincidencia).
F3: Como organizador, quiero aprobar todos los tickets reservados de un comprador en una rifa para confirmar su compra.
F4: Como organizador, quiero liberar (rechazar) todos los tickets de un comprador en una rifa para devolverlos a disponibilidad.
F5: Como organizador, quiero seleccionar qué tickets apruebo para que el resto se libere automáticamente.

[A] 1. Organizador (primario) → listar, buscar, aprobar, liberar
    2. Comprador (secundario) → reserva desde la página de la rifa
    3. Feature de Tickets (interno) → transiciones de estado
    4. Página de la rifa (interno) → genera reservas

[D] Organizador:
    [R] Listar últimos 30 compradores con tickets reservados
    [R] Buscar compradores por nombre o teléfono
    [R] Ver tickets reservados de un comprador (por rifa)
    [T] Aprobar todos los tickets de un comprador en una rifa
    [T] Liberar todos los tickets de un comprador en una rifa
    [T] Aprobar tickets seleccionados
    [T] Liberar tickets no seleccionados (el resto)
Sistema interno (Feature de Tickets):
    [Validación] Validar que el ticket esté en estado reservado
    [Acción] Cambiar estado del ticket (→ aprobado | → disponible)

[E] Comprador (id, nombre, teléfono, fechaReservaMásReciente)
Ticket (id, número, estado: disponible|reservado|aprobado,
      fechaReserva, idComprador?, idRifa)
Rifa (id, título, idOrganizador) · Organizador (id, nombre)
→ Decisión: "Reserva" NO es entidad; es Ticket en estado reservado.

[R] R001: Solo se muestran tickets de rifas del organizador (Restricción)
R002: Lista = últimos 30 compradores por fecha de reserva desc (Restricción)
R003: Búsqueda por nombre o teléfono, contiene, ignora mayúsculas (Validación)
R004: Solo se aprueban/liberan tickets en estado reservado (Validación)
R005: Un ticket aprobado no se aprueba ni libera de nuevo (Consistencia)
R006: Al liberar, el ticket vuelve a disponible (Transición)
R007: Un comprador puede reservar de varias rifas del organizador (Restricción)
R008: Aprobar selección = aprobar elegidos + liberar resto atómicamente (Consistencia)
R009: Aprobar solo cambia estado a aprobado (Flujo)
```

Checklist de autoevaluación FADER

- ☐ Cada enunciado sigue “Como [actor], quiero [acción] para [valor]”
- ☐ Cada actor tiene ≥1 operación que le pertenece · ninguno “porque sí”
- ☐ Cada operación pasa el test de atomicidad · sin “y” en el nombre

- ☐ Entidades del mundo real, atributos esenciales, relaciones definidas
- ☐ Cada regla: código único + categoría + mensaje de error donde aplique
- ☐ Revisión cruzada: regla↔operación · operación↔actor · entidad↔regla

2 · Mapeo FADER → Capas de Clean Architecture (3 min)

Regla fundamental: cada elemento de FADER se mapea a **una y solo una** capa.

Elemento FADER	Capa	Ejemplo / archivo
Entidades	DOMAIN entities/	product.dart
Reglas (validación)	DOMAIN usecases/	dentro de su UseCase
Operaciones (casos de uso)	DOMAIN usecases/	add_product_to_cart.dart
Contratos (interfaces)	DOMAIN repositories/ + services/	cart_repository.dart
Fuentes de datos	DATA datasources/	cart_remote_data_source.dart
Modelos (DTO / JSON)	DATA models/	cart_model.dart
Implementación de contratos	DATA repositories/	cart_repository_impl.dart
Estados de UI	PRESENTATION cubit/	cart_state.dart
Widgets	PRESENTATION pages/ + widgets/	cart_page.dart

Test de capa: ¿podría este código vivir en un proyecto Dart sin Flutter? **Sí → Domain** · No → otra capa.

Errores comunes:

Error	Solución
fromJson() en entidad de Domain	Va en Model (DATA)
UseCase que llama supabase.from(...)	Usa el Repository; no sabe de dónde vienen los datos
Estado del Cubit con flags de widgets	El estado describe datos, no widgets
Regla de negocio en un Widget	Va en UseCase
Interface de Repository en DATA	La interface pertenece a DOMAIN
Pasar userId en cada método	Inyecta UserSession en el RepositoryImpl

3 · Contract-First Design (3 min)

Define las **interfaces públicas** de cada capa **antes** de implementar. El “qué” antes del “cómo”.

Los 3 contratos clave

- ```
1 · REPOSITORY (Domain → Data) Define qué datos necesita el dominio.
Future<Either<Failure, Cart>> getCart()
Future<Either<Failure, Cart>> addProduct(Product p, int qty)

2 · DATASOURCE (Data → API/BD) Operaciones de bajo nivel.
Future<CartModel> fetchCart(String userId)
Future<void> cacheCart(String userId, CartModel cart)

3 · USECASE (Presentation → Domain) Operaciones de negocio para la UI.
Future<Either<Failure, Cart>> call({required Product p, required int qty})
```

**Validación del contrato**

- **Independiente de tecnología:** sin “Supabase”, “HTTP”, “SQL”.
- **Completo:** cubre toda la descomposición FADER.
- **Consistente:** mismo patrón en todos los métodos.
- **Congelar:** no cambia por decisiones de implementación. Documenta con ADR.

ADR — Architecture Decision Record

```
ADR-001: Contrato del Repositorio
Contexto - ¿Por qué decidimos?
Decisión - ¿Qué elegimos y cómo?
Consecuencias - Positivas y negativas
Alternativas - 1..n consideradas y por qué se descartaron
```

4 · Flujo de Datos (3 min)

Cada operación sigue una **forma de U**: UI → Domain → Data → vuelta a UI.

```
USUARIO -> Widget -> Cubit(evento) -> UseCase(valida reglas)
 -> Repository(interface) -> RepositoryImpl -> DataSource(remote/local)
 -> API/Caché -> DataSource -> Model -> RepositoryImpl -> Entity
 -> UseCase -> Right(Entity) -> Cubit -> fold(estado) -> Widget(rebuild)
```

**Tipos por capa — nunca pases un Model a la UI ni una Entity al DataSource:**

| Capa       | Entrada       | Salida                  |
|------------|---------------|-------------------------|
| Widget     | UserAction    | Event                   |
| Cubit      | Event         | call UseCase            |
| UseCase    | params        | Either<Failure, Entity> |
| Repository | Domain params | Either<Failure, Entity> |
| DataSource | Model params  | Model                   |
| API/BD     | JSON/SQL      | JSON/Row                |

**Flujo de errores (abajo → arriba):** API → DataSource captura excepción → `Failure` → RepoImpl → UseCase → Cubit `fold()` → mensaje → Snackbar. **Regla:** nunca dejes que una excepción técnica (HTTP 500, timeout, SQL) llegue a la UI.

5 · Estimación de Complejidad (2 min)

Descompón en 5 componentes y estima cada uno.

| Letra | Componente    | Ejemplo                 |
|-------|---------------|-------------------------|
| F     | Frontend      | Pantalla de perfil      |
| A     | API / Backend | CRUD de usuarios        |
| D     | Data          | User model + repository |
| E     | Estados       | AuthCubit               |
| R     | Requisitos    | Login required          |

**Proceso:** 1) descompón la feature · 2) estima cada componente · 3) suma **+30% buffer** (y +20% testing).

| Nivel    | Tiempo | Señales                            |
|----------|--------|------------------------------------|
| Simple   | 1-3h   | Solo UI, sin backend               |
| Media    | 4-8h   | UI + 1-2 llamadas, 1 Cubit         |
| Compleja | 8-16h  | Backend + realtime, varios estados |
| Épica    | 16-40h | Multi-pantalla, integraciones      |

6 · Plantilla “Toma y llena” — para cada feature futura

Copia este bloque, llénalo y tendrás el diseño completo.

```
Feature: [Nombre]

FADER
[F] Como [actor], quiero [acción] para [valor].
[A] 1. [actor] (tipo) → [permisos]
[D] - [R] [operación] - [T] [operación]
[E] [Entidad] (atributos esenciales)
[R] R001: [regla] (categoría) · mensaje: "[...]"

Mapeo
Elemento FADER	Capa	Archivo
Entidad	DOMAIN entities/	x.dart
Operación	DOMAIN usecases/	x.dart
Contrato Repo	DOMAIN repositories/	x_repository.dart
Modelo	DATA models/	x_model.dart
DataSource	DATA datasources/	x_remote_data_source.dart
RepoImpl	DATA repositories/	x_repository_impl.dart
Estado	PRESENTATION cubit/	x_state.dart
Página	PRESENTATION pages/	x_page.dart

Contratos (3)
```dart
// Repository, DataSource, UseCase → Future<Either<Failure, T>>
```

Flujo

USUARIO → Widget → Cubit → UseCase → Repository → DataSource → API → vuelta.

Estimación

F: [h] · A: [h] · D: [h] · E: [h] · R: [h] → Base [X]h · +30% buffer → Total [Y]h ```

Anexo · Índice del módulo (para profundizar)

Necesitas	Archivo
Teoría FADER completa (event storming, user story mapping, second pass)	01-descomposicion-feature.md
Práctica de descomposición	01a-practica-carrito.md
Mapeo a capas en detalle	02-mapeo-capas.md
Práctica de mapeo	02a-practica-carrito-capas.md
Contract-First en detalle	03-contratos-primero.md
Práctica de contratos	03a-practica-carrito-contratos.md
Flujo de datos en detalle	04-flujo-datos.md
Práctica de flujo	04a-practica-carrito-flujo.md
Casos integradores (Reservas, E-Learning, Facturación, Delivery)	05*.md
Estimación de complejidad	15-estimacion-complejidad.md
Checklists y plantillas del flujo “sin IA”	trabajar-sin-ia/13-checklists-y-plantillas.md
Hoja FADER de ejemplo (Buyers)	05e-caso-buyers-fader.md

7 · Bibliografía y Fuentes

Fuentes que fundamentan cada decisión del módulo. Versión condensada — la completa está en [BIBLIOGRAFIA.md](#).

Libros base		
Libro	Autor (año)	Aportación al módulo
Clean Architecture	Robert C. Martin (2017)	Los 3 capas, la regla de dependencia, los UseCases como comportamiento

Clean Architecture	Robert C. Martin (2017)	Las 3 capas, la regla de dependencia, los usecases como orquestadores
Domain-Driven Design	Eric Evans (2003)	Lenguaje ubicuo, agregados, entidades vs value objects, reglas en el dominio
Growing Object-Oriented Software, Guided by Tests	Freeman & Pryce (2009)	Contract-First (diseño de afuera hacia adentro), mocks como herramienta de diseño
Refactoring	Martin Fowler (1999/2018)	Mejorar el diseño existente sin romperlo; el costo de no diseñar
Design Patterns (GoF)	Gamma, Helm, Johnson, Vlissides (1994)	Patrón Repository, Adapter, Strategy en Clean Architecture

Patrones y metodologías

Patrón	Origen	En el módulo
Contract-First Design	Bertrand Meyer (1986) — Design by Contract	Cada contrato define responsabilidades, parámetros, retornos y errores
ADR (Architecture Decision Records)	Michael Nygard (2011)	Documentar decisiones: Contexto → Decisión → Consecuencias → Alternativas
Result Pattern / Railway Oriented Programming	Scott Wlaschin (2014)	<code>Either<Failure, Success></code> en vez de excepciones
fpdart	Sandro Maglione	Implementación de <code>Either</code> , <code>Option</code> , <code>TaskEither</code> en Dart

Metodología FADER

- Origen:** invención propia para este módulo (2026). Adapta: User Story Mapping (Patton) · análisis de actores UML / Use Case 2.0 (Jacobson) · Event Storming (Brandolini) · DDD (Evans) · Business Rules.

Paquetes referenciados

Paquete	Uso
flutter_bloc	Manejo de estado (Presentation)
equatable	Comparación de objetos
get_it / injectable	Inyección de dependencias
fpdart	Either, Option, TaskEither
supabase_flutter	Backend como servicio
freezed (alternativa)	Sealed classes generadas

Tiempo objetivo por feature: ~20 min · Regla de oro: piensa primero, codifica después.